

Narratives and queries

A program as a sheaf and a cosheaf, and the temporal logic that follows

Niu, Osgood, Zelko and Srinivasan describe time-varying data as *narratives*: sheaves on the poset of closed intervals of a timeline. An intervention program is one. Its *persistent* narrative sends an interval to the atoms in force throughout it; its *cumulative* narrative sends an interval to the atoms in force at some time in it. This vignette evaluates both, checks the gluing equations that make them a sheaf and a cosheaf, and uses Catlab's subobject logic to ask temporal questions about a policy timeline.

```
using CategoricalInterventions
```

A policy timeline

```
space = TargetSpace()
declare!(space, :contact, Affine(Multiplicative()); value_type=Float64)
declare!(space, :masks, Affine(Multiplicative()); value_type=Float64)
declare!(space, :testing, Affine(Additive()); value_type=Float64)

policy = @interventions space begin
    lockdown_1 = scale(:contact, 0.4; during=10..31)
    lockdown_2 = scale(:contact, 0.6; during=35..50)
    masking     = scale(:masks, 0.7; during=20..60)
    surge       = add(:testing, 500.0; during=25..45)
end
explain(policy)
```

Epochs:

```
[10, 20) lockdown_1
        contact: scale(0.4)
[20, 25) lockdown_1, masking
```

```

        contact: scale(0.4)
        masks: scale(0.7)
[25, 31) lockdown_1, masking, surge
        contact: scale(0.4)
        masks: scale(0.7)
        testing: add(500.0)
[31, 35) masking, surge
        masks: scale(0.7)
        testing: add(500.0)
[35, 45) lockdown_2, masking, surge
        contact: scale(0.6)
        masks: scale(0.7)
        testing: add(500.0)
[45, 50) lockdown_2, masking
        contact: scale(0.6)
        masks: scale(0.7)
[50, 60) masking
        masks: scale(0.7)

```

Persistent and cumulative narratives

Evaluate a narrative on a closed interval [lo, hi] or at a point.

```

A = persistent(policy)
C = cumulative(policy)
ids(x) = [a.id for a in x]
(throughout_25_40 = ids(A(25, 40)), sometime_25_40 = ids(C(25, 40)), at_33 = ids(A(33)))

```

```

(throughout_25_40 = [:masking, :surge], sometime_25_40 = [:lockdown_1, :lockdown_2, :masking]

```

Between the two lockdowns nothing acts on contact, so contact is under intervention at day 30 and at day 36 but not throughout [30, 36].

```

throughout(policy, :contact, (30, 36)), sometime(policy, :contact, (30, 36))

```

```

(false, true)

```

The gluing equations

For $a \leq p \leq b$, the persistent narrative satisfies $A[a, b] = A[a, p] \cup A[p, b]$ and the cumulative one $C[a, b] = C[a, p] \cup C[p, b]$ with $C[a, p] \cap C[p, b] = C[p, p]$. These are the sheaf and cosheaf conditions of the interval site; here they are checked on a few triples.

```
triples = [(10, 20, 31), (25, 33, 40), (0, 35, 60), (30, 30, 36)]
map(triples) do (a, p, b)
  persistent_ok = Set(A(a, b)) == intersect(Set(A(a, p)), Set(A(p, b)))
  cumulative_ok = Set(C(a, b)) == union(Set(C(a, p)), Set(C(p, b))) &&
    intersect(Set(C(a, p)), Set(C(p, b))) == Set(C(p, p))
  (a, p, b) => (persistent_ok, cumulative_ok)
end
```

```
4-element Vector{Pair{Tuple{Int64, Int64, Int64}, Tuple{Bool, Bool}}}:
 (10, 20, 31) => (1, 1)
 (25, 33, 40) => (1, 1)
 (0, 35, 60) => (1, 1)
 (30, 30, 36) => (1, 1)
```

The second cumulative equation depends on supports being convex: an atom that meets $[a, p]$ and $[p, b]$ must contain p .

A discrete window and its truth values

On a finite grid, a target-level narrative is a graph over the path graph: a vertex for “under intervention at t ” and an edge for “under intervention throughout $[t, t']$ ”. A program is a subobject of the window’s ambient graph. Each unit interval then carries one of the five local truth values of the paper’s subobject classifier.

```
window = NarrativeWindow([Target(:contact), Target(:masks), Target(:testing)], 0:5:60)
s = subobject(window, policy)
describe(s)
```

```
contact:
 [0, 5]  not in force
 [5, 10] not in force at the start, in force by the end
 [10, 15] in force throughout
 [15, 20] in force throughout
 [20, 25] in force throughout
```

```

[25, 30] in force throughout
[30, 35] in force at both ends but lifted in between
[35, 40] in force throughout
[40, 45] in force throughout
[45, 50] in force at the start, lifted before the end
[50, 55] not in force
[55, 60] not in force
masks:
[0, 5] not in force
[5, 10] not in force
[10, 15] not in force
[15, 20] not in force at the start, in force by the end
[20, 25] in force throughout
[25, 30] in force throughout
[30, 35] in force throughout
[35, 40] in force throughout
[40, 45] in force throughout
[45, 50] in force throughout
[50, 55] in force throughout
[55, 60] in force at the start, lifted before the end
testing:
[0, 5] not in force
[5, 10] not in force
[10, 15] not in force
[15, 20] not in force
[20, 25] not in force at the start, in force by the end
[25, 30] in force throughout
[30, 35] in force throughout
[35, 40] in force throughout
[40, 45] in force at the start, lifted before the end
[45, 50] not in force
[50, 55] not in force
[55, 60] not in force

```

The value “in force at both ends but lifted in between” appears on [30, 35] for contact: the first lockdown ends at 31 and the second starts at 35.

```
truth_value(s, Target(:contact), 7)
```

```
:both_ends_not_throughout
```

Heyting operations

Subobjects of a window form a Heyting algebra, so programs can be combined logically. Each target is its own row of the window, so to compare two policies as temporal propositions they are first placed on a common row by renaming their targets to `:policy`. Then `masks` `lockdown` asks “wherever masking is in force, is a lockdown in force?”

```
masks_p      = pushforward(Program(policy.atoms[3:3]; space=space), Dict(:masks => :policy))
lockdown_p   = pushforward(Program(policy.atoms[1:2]; space=space), Dict(:contact => :policy))
testing_p    = pushforward(Program(policy.atoms[4:4]; space=space), Dict(:testing => :policy))

row = NarrativeWindow([Target(:policy)], 0:5:60)
M, L, T = subobject(row, masks_p), subobject(row, lockdown_p), subobject(row, testing_p)
implication = M  L
[(row.times[k], row.times[k + 1]) => truth_value(implication, Target(:policy), k) for k in 1
```

12-element Vector{Pair{Tuple{Int64, Int64}}}:
12-element Vector{Pair{Tuple{Int64, Int64}}}:
(0, 5) => :throughout
(5, 10) => :throughout
(10, 15) => :throughout
(15, 20) => :throughout
(20, 25) => :throughout
(25, 30) => :throughout
(30, 35) => :both_ends_not_throughout
(35, 40) => :throughout
(40, 45) => :throughout
(45, 50) => :start_only
(50, 55) => false
(55, 60) => :end_only

The implication is true where there is no masking, true where both hold, “in force at both ends but lifted in between” on `[30, 35]` where masking continues across the gap between the two lockdowns, and false from day 50 when masks are worn without a lockdown. The edges of that failure show the half-open convention: on `[45, 50]` the second lockdown is in force at 45 but not at 50 (“in force at the start, lifted before the end”), and on `[55, 60]` the implication is vacuously true again at 60, when masking itself ends. Meet and negation read the same way: masking together with surge testing holds throughout `[25, 30]`; the negation of masking holds before masking begins and fails during it.

```
(masks_and_testing = truth_value(M  T, Target(:policy), 6),
no_masks_early     = truth_value(¬M, Target(:policy), 2),
no_masks_late      = truth_value(¬M, Target(:policy), 6))
```

```
(masks_and_testing = :throughout, no_masks_early = :throughout, no_masks_late = false)
```

Scope: these queries operate on the *specification* of a program, restricted to a finite window of a grid. They say nothing about simulation output.

What the category theory bought

The persistent and cumulative narratives satisfy the sheaf and cosheaf gluing equations (`persistent_glue`, `cumulative_union`, `cumulative_inter_eq_point`), and on a point both reduce to the active set (`persistent_eq_active_of_point`). Because the persistent narrative is a presheaf on the zigzag category, the window graph built from it is a valid discrete narrative in the sense of Niu et al., which is what licenses handing it to Catlab's subobject classifier. The topos structure is Catlab's; the package supplies the encoding and the proof that it is sound.